

Docker Troubleshooting Guide

1. Docker Basics

Docker packages an application and its dependencies into containers. A container is created from a Docker image. Containers are isolated but share the host kernel.

Common objects are **images, containers, networks, volumes, and Compose services**.

2. Container Status

Use the following commands to determine whether a container is running or stopped.

```
docker ps
docker ps -a
```

A container shown by **docker ps -a** but not by **docker ps** is normally stopped.

3. Logs and Failures

Container logs are often the first evidence when an application fails.

```
docker logs <container>
docker logs --tail 100 <container>
docker logs -f <container>
```

For a stopped container, also check its exit code.

```
docker inspect <container> --format '{{.State.ExitCode}}'
```

Look for startup errors, connection failures, missing configuration, permission errors, and dependency failures.

4. CPU and Memory

Use Docker's real-time statistics to identify resource-heavy containers.

```
docker stats
docker top <container>
```

High CPU can be caused by application bugs, infinite loops, background jobs, unexpected traffic, or runaway processes. High memory can be caused by leaks, large workloads, too many workers, or insufficient host memory.

5. Networking and Ports

Check Docker networks and published ports when an application cannot be reached.

```
docker network ls
docker network inspect <network>
docker port <container>
sudo ss -lntp
```

Containers in the same Docker network can normally communicate using service/container names. A published port maps a host port to a container port.

6. Docker Compose

Compose manages multi-container applications defined in a Compose file.

```
docker compose up -d
docker compose ps
docker compose logs -f <service>
docker compose restart <service>
docker compose down
```

If a service fails, check its status, logs, dependencies, ports, and configuration before restarting it.

7. Safe Troubleshooting

Read-only investigation should normally happen before remediation.

Useful read-only commands include **docker ps**, **docker logs**, **docker inspect**, **docker stats**, **docker top**, **docker network inspect**.

Potentially disruptive commands include **docker restart**, **docker stop**, **docker kill**, **docker rm**. Destructive cleanup such as removing volumes should require particular care.

Recommended workflow: **observe** → **collect evidence** → **identify root cause** → **obtain approval if needed** → **remediate** → **verify**.